

Welcome back! Pichli class mein humne badi documents ko chhote **Chunks** mein todna seekha tha. Lekin ek sawal hai: Agar aapke paas 10,000 chunks hain, toh LLM ko kaise pata chalega ki user ke question ka answer kaunse specific chunk mein chhipa hai?

Yahin se GenAI ka sabse powerful concept shuru hota hai: **RAG (Retrieval-Augmented Generation)**. Aaj hum data ko "searchable" banana seekhenge.

Module 1: The Foundation of Search (Topics 1, 2, 3)

1. Why searching documents is difficult

Traditional search systems (jaise Ctrl+F ya SQL LIKE queries) **Keyword Matching** par kaam karte hain.

- **Real-world Analogy:** Agar aap search karte ho "*sick leave*", par company ki HR policy PDF mein likha hai "*medical absence*", toh traditional search fail ho jayega kyunki exact words match nahi huye. GenAI ko **Semantic Search** (Meaning/Bhavna) samajhni hoti hai, exact words nahi.

2. What is Retrieval? & 3. RAG architecture overview

Retrieval ka matlab hai user ka question padhna aur hazaron chunks mein se top 2-3 sabse relevant chunks nikal kar lana, chahe unke words alag hon par meaning same ho.

RAG (Retrieval-Augmented Generation) wo architecture hai jahan hum pehle apne data ko "Retrieve" (search) karte hain, aur fir us data ko prompt mein daal kar LLM se answer "Generate" karwate hain.

Module 2: The Math Behind the Magic (Topics 4, 5)

4. Embeddings Concept

Computer text (English/Hindi) nahi samajhta, wo numbers samajhta hai.

Embeddings ek machine learning model hota hai jo kisi bhi text (word, sentence, ya poore chunk) ko ek lambi numbers ki list (Array/Vector) mein convert kar deta hai (e.g., [0.12, -0.45, 0.89...]).

- **Real-world Analogy:** Socho words ko ek bade 3D map par GPS coordinates diye gaye hain. "Dog" aur "Puppy" alag words hain, par unke GPS coordinates map par ek doosre ke bohot paas honge. "Car" ka coordinate bohot door hoga. Isko Semantic Space kehte hain.

5. Vector databases introduction

Jab humare paas hazaron chunks ke thousands of GPS coordinates (Vectors) aa jate hain, toh unhe standard SQL database mein save nahi kiya ja sakta. Hamein ek special **Vector Database** (jaise FAISS, ChromaDB, Pinecone) chahiye hota hai. Yeh DBs specifically in vectors ko store aur fast search karne ke liye design kiye gaye hain.

Let's look at an interactive visualization to understand how this search actually works in the background before we jump to the code.

Module 3: Retrievers in LangChain (Topics 6, 7, 8, 9)

6. What is a Retriever?

LangChain mein Retriever ek simple interface hai. Aap isko ek string (query) dete ho, aur yeh aapko list of Document objects wapas lauta deta hai.

7. VectorStoreRetriever working & 8. Similarity search

Jab aap ek Vector Database banate ho, toh LangChain usko easily ek VectorStoreRetriever mein convert kar deta hai.

- **Internal Working:** Jab user question poochta hai, system pehle us question ko Embedding (Vector/GPS coordinate) mein convert karta hai. Phir Vector DB mein **Similarity Search** (Cosine Similarity ya Euclidean Distance) run karta hai, jo check karta hai ki is question ke GPS coordinate ke sabse nazdeek kaunse Chunks ke coordinates hain.

9. Top-K retrieval

Yeh ek parameter k hota hai jo batata hai ki database se kitne results wapas lane hain. Agar k=3, toh DB sabse close 3 chunks lauta dega.

Module 4: The Ultimate RAG Pipeline (Topic 10 & Application)

Document → Chunk → Embedding → Vector DB → Retriever → LLM

Architecture flow diagram:

1. [PDF Document] -> TextLoader
 2. [Raw Text] -> RecursiveCharacterTextSplitter -> [Chunk 1, Chunk 2...]
 3. [Chunks] -> OpenAIEmbeddings -> [Vector 1, Vector 2...]
 4. [Vectors] -> Saved inside FAISS Vector DB
-
5. User asks: "What is the timeout error?" -> Converted to Query Vector
 6. FAISS searches and returns Top 2 nearest Vectors (Chunks)
 7. [Retriever] -> Passes the 2 Chunks to PromptTemplate
 8. Prompt + Chunks -> LLM -> Beautiful Text Answer!

Complete Python Code: Small Document Q&A System

```
import os
from dotenv import load_dotenv
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

# -----
# STEP 1: Load and Chunk Data
# -----
dummy_doc = Document(page_content="""
HealthBuddy v2.0 System Architecture:
The backend is built using Spring Boot microservices.
Patient medical records are stored securely in a Neo4j graph database.
For critical triage, we use CustomCNN models to detect anomalies in X-Rays.
System timeout errors usually occur if the Neo4j connection pool is exhausted.
""")

splitter = RecursiveCharacterTextSplitter(chunk_size=100, chunk_overlap=20)
chunks = splitter.split_documents([dummy_doc])
print(f"Created {len(chunks)} chunks.")

# -----
# STEP 2: Create Embeddings & Store in Vector DB (FAISS)
# -----
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
# This line creates vectors for all chunks and stores them in RAM
vector_db = FAISS.from_documents(chunks, embeddings)

# -----
```

```

# STEP 3: Setup the Retriever
# -----
# search_kwargs={"k": 2} means bring back the top 2 matching chunks
retriever = vector_db.as_retriever(search_kwargs={"k": 2})

# -----
# STEP 4: Build the Modern LCEL RAG Chain
# -----
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
parser = StrOutputParser()

template = """Answer the user's question based ONLY on the following context:
Context: {context}

Question: {question}
Answer: """
prompt = PromptTemplate.from_template(template)

# The Magic LCEL Chain
# RunnablePassthrough() takes the raw input string ("Why is there a timeout?")
# and passes it simultaneously to the 'question' variable AND to the 'retriever'
rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | llm
    | parser
)

# -----
# STEP 5: Execute (Dry Run)
# -----
print("\nUser: Why does the system timeout?")
answer = rag_chain.invoke("Why does the system timeout?")
print(f"AI: {answer}")

```

Code Execution Dry Run

1. **Chunks:** Text 4-5 chhote tukdon mein kat gaya.
2. **Embeddings:** OpenAIEmbeddings ne OpenAI ke API ko call kiya aur un chunks ko numbers mein badal kar FAISS ko de diya.
3. **LCEL Pipeline:** Jab humne `.invoke("Why does the system timeout?")` run kiya, `RunnablePassthrough` ne us string ko pakda. Usne wo string retriever ko di. Retriever FAISS ke paas gaya, Vector matching ki, aur timeout error wala chunk utha laya. Phir prompt populate hua aur LLM ne human-readable answer diya.

Sample Output

Created 5 chunks.

User: Why does the system timeout?

AI: System timeout errors usually occur if the Neo4j connection pool is exhausted.

Review Section

Common Mistakes Beginners Make

1. **Embedding Model Mismatch:** Agar VectorDB banate waqt text-embedding-3-small use kiya hai, aur retriever search karte waqt koi aur model (like text-embedding-ada-002) use karta hai, toh DB completely garbage results dega. Maps (dimensions) match hone zaroori hain.
2. **Not verifying retrieved chunks:** Hamesha print karke dekho ki retriever sahi chunks laa raha hai ya nahi: `print(retriever.invoke("your question"))`. Agar chunks galat hain, toh LLM answer humesha galat hi dega.
3. **Forgetting RunnablePassthrough:** LCEL mein dictionary input ko pass karne ka yahi standard tarika hai.

Interview Questions & Answers

Q: What is the exact difference between a Vector Store and a Retriever in LangChain?

Answer: A Vector Store (like FAISS or Chroma) is the actual database that stores embeddings and performs the mathematical similarity search. A Retriever is a higher-level LangChain interface that abstracts away the database complexity; it simply takes a string query and returns a list of Documents. A Vector Store can act as a Retriever using `.as_retriever()`.

Q: If you want the Retriever to filter documents by metadata (e.g., only search PDFs from the year 2025), how would you do it?

Answer: You pass a filter dictionary inside the `search_kwargs`. For example:
`vectorstore.as_retriever(search_kwargs={"k": 3, "filter": {"year": "2025"}})`

Practice Exercises

- **Task 1:** Take the code above and add a new dummy document about a random topic (e.g., "The weather in Bengaluru is sunny"). Run the system and ask about the weather. Does it retrieve the right chunk?
- **Task 2:** Change `k=1` in `search_kwargs`. Now ask a complex question that requires

information from two different sentences in the text. Watch how the LLM fails because it didn't get enough context.

Aapne RAG ka core engine (Embeddings + Vector Search) master kar liya hai! Abhi humara chain purani history yaad nahi rakh raha hai. Kya hum next class mein

ConversationalRetrievalChain banayein, taaki system PDFs par answer bhi de sake aur pichle follow-up questions ko bhi yaad rakh sake?